

Лабораторная работа № 1
Исследование модели экспоненциального роста
Исаева Зарина Исмайлбековна
2026-09-05

Аннотация

Исследована задача Коши $du/dt = \alpha u$ для одного набора параметров и серии значений коэффициента роста. Численное решение получено методом Tsit5 и сопоставлено с аналитической формулой. Исходные программы оформлены как литературный код, из которого сформированы чистые Julia-сценарии, Jupyter-ноутбуки и Quarto-документы. Результат проверен автоматическими тестами и сохранён в воспроизводимой структуре проекта.

Содержание

1. Паспорт работы
2. Цель и задания
3. Математическая постановка
4. Организация вычислительного проекта
5. Реализация модели
6. Полный базовый сценарий
7. Полный параметрический сценарий
8. Литературное программирование и производные форматы
9. Проверка и воспроизводимость
10. Учёт изменений
11. Выводы
12. Источники

Паспорт работы

Поле	Значение
Автор	Исаева Зарина Исмайлбековна
Студенческий билет	1132232882
Дисциплина	Имитационное моделирование
Лабораторная работа	№ 1
Тема	Модель экспоненциального роста

1. Цель и задания

Цель работы — освоить воспроизводимую организацию вычислительного проекта на Julia и исследовать свойства модели экспоненциального роста.

Для достижения цели выполнены следующие задания:

1. создан отдельный Julia-проект с закреплёнными зависимостями;
2. реализован базовый сценарий решения задачи Коши;
3. проведена серия экспериментов для пяти значений параметра α ;
4. из литературных исходников получены скрипты, QMD и IPYNB;
5. выполнены ноутбуки, сформированы таблицы и графики;
6. численные результаты проверены аналитически и автоматическими тестами;
7. подготовлены отчёт, презентация и журнал изменений.

2. Математическая постановка

Пусть $u(t)$ — величина, изменяющаяся со скоростью, пропорциональной своему текущему значению. Тогда модель имеет вид

$$\frac{du}{dt} = \alpha u, u(0) = u_0, (1)$$

где u_0 — начальное значение, а α — постоянный коэффициент роста. После разделения переменных получаем аналитическое решение

$$u(t) = u_0 e^{\alpha t}. (2)$$

Время удвоения определяется условием $u(T_2) = 2u_0$:

$$T_2 = \frac{\ln 2}{\alpha}, \alpha > 0. (3)$$

Для базового опыта приняты $u_0=1$, $\alpha=0,3$ и $t \in [0; 10]$. Значения решения сохраняются с шагом 0,1.

3. Организация вычислительного проекта

Каталог лабораторной работы разделён по назначению: модуль модели находится в `src`, литературные сценарии — в `scripts`, результаты — в `data` и `plots`, а производные документы — в `notebooks` и `markdown`. Отчёт и презентация собираются из самостоятельных QMD-файлов.

```
rhktrlmnd@Mac 2026-1--study--simulation-modeling % find labs -maxdepth 2 -type d | sort
labs
labs/lab01
labs/lab01/image
labs/lab01/presentation
labs/lab01/project
labs/lab01/report
labs/lab02
labs/lab02/presentation
labs/lab02/report
labs/lab03
labs/lab03/presentation
labs/lab03/report
labs/lab04
labs/lab04/presentation
labs/lab04/report
labs/lab05
labs/lab05/presentation
labs/lab05/report
labs/lab06
labs/lab06/presentation
labs/lab06/report
labs/lab07
labs/lab07/presentation
labs/lab07/report
labs/lab08
labs/lab08/presentation
labs/lab08/report
rhktrlmnd@Mac 2026-1--study--simulation-modeling %
```

Рисунок 1: Структура каталогов лабораторной работы

Пакеты и их точные версии записаны в `Project.toml` и `Manifest.toml`. Поэтому после клонирования репозитория среда восстанавливается командой `julia --project=. -e 'using Pkg; Pkg.instantiate()'`.

```
name = "project"
uid = "5e6fc9a5-189d-46e2-b024-9dd395edfa9f"
authors = ["rhktrlmnd"]
version = "1.0.0"

[deps]
BenchmarkTools = "664b89f9-dd63-53aa-95a3-8c0b28fa8ba"
CSV = "336ed68f-9bac-5cae-87d4-7b16caf5d00b"
DataFrames = "a93c6f90-e57d-5684-b7b6-d8193f3e46c8"
DifferentialEquations = "0c4d8022-0b33-5123-abaf-578d42b7fbaa"
DrWatson = "634d3b9d-a77a-5ddf-bec9-22491ea816e1"
IJulia = "7073ff75-c697-5162-941a-fcdaed2a7d2a"
JLD2 = "83883b0b-8acc-5ee0-8aee-3f567fa3019"
Literate = "98b81ad-f1c9-55d5-8b28-4c87d4299306"
Plots = "91a5bcdd-55d7-5caf-9e0b-5208859cae80"
Quarto = "d7167be5-f61b-4dc9-b75c-ab62374668c5"

[compat]
julia = "1.11"
rhktrlmnd@Mac 2026-1--study--simulation-modeling %
```

Рисунок 2: Проверка закреплённых зависимостей проекта

Компонент	Назначение
DifferentialEquations.jl	постановка задачи Коши и интегрирование методом Tsit5
DrWatson.jl	пути проекта и параметрические эксперименты
DataFrames.jl, CSV.jl, JLD2.jl	табличные и бинарные результаты
Plots.jl	графики траекторий и зависимостей
Literate.jl, IJulia.jl	генерация QMD, Julia-скриптов и ноутбуков
BenchmarkTools.jl	измерение времени решения
Quarto	сборка отчёта, презентации и HTML-документации

4. Реализация модели

4.1 Общая функция правой части

Математическая часть вынесена в `src/exponential_growth.jl`. Одна функция принимает как скалярный параметр, так и именованный кортеж, поэтому используется в обоих сценариях.

```
"""Правая часть уравнения экспоненциального роста `du/dt = αu`."""
function exponential_growth!(du, u, p, t)
```

```

alpha = p isa NamedTuple ? p.alpha : p
du[1] = alpha * u[1]
return nothing
end

analytic_solution(u0, alpha, t) = u0 * exp(alpha * t)

function doubling_time(alpha)
    alpha > 0 || throw(ArgumentError("alpha must be positive"))
    return log(2) / alpha
end

```

Листинг 1 — Полный модуль модели `src/exponential_growth.jl`.

4.2 Базовый эксперимент

В литературном сценарии `01_exponential_growth.jl` задача описана вместе с поясняющим текстом. Для численного решения создан `ODEProblem`, а затем вызван адаптивный решатель `Tsit5`.

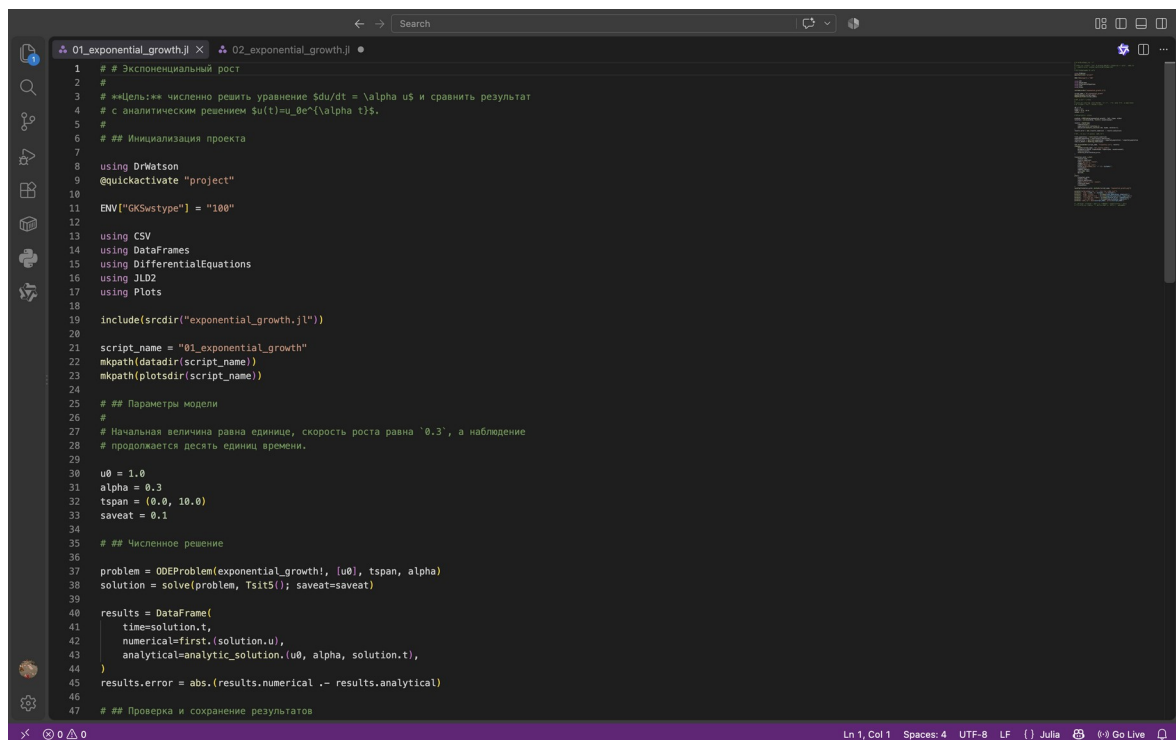


Рисунок 3: Фрагмент литературного сценария базового опыта

Ниже непосредственно подключён полный QMD-фрагмент, автоматически полученный из литературного исходника `scripts/01_exponential_growth.jl`. В нём сохранены инициализация проекта, параметры, решение,

аналитическая проверка, запись CSV и JLD2, построение графика и вывод контрольных величин.

5. Экспоненциальный рост

Цель: численно решить уравнение $du/dt = \alpha u$ и сравнить результат с аналитическим решением $u(t) = u_0 e^{\alpha t}$.

5.1 Инициализация проекта

```
using DrWatson
@quickactivate "project"

ENV["GKSwstype"] = "100"

using CSV
using DataFrames
using DifferentialEquations
using JLD2
using Plots

include(scrdir("exponential_growth.jl"))

script_name = "01_exponential_growth"
mkpath(datadir(script_name))
mkpath(plotsdir(script_name))
```

5.2 Параметры модели

Начальная величина равна единице, скорость роста равна 0.3, а наблюдение продолжается десять единиц времени.

```
u0 = 1.0
alpha = 0.3
tspan = (0.0, 10.0)
saveat = 0.1
```

5.3 Численное решение

```
problem = ODEProblem(exponential_growth!, [u0], tspan, alpha)
solution = solve(problem, Tsit5(); saveat=saveat)

results = DataFrame(
```

```

time=solution.t,
numerical=first(solution.u),
analytical=analytic_solution(u0, alpha, solution.t),
)
results.error = abs(results.numerical - results.analytical)

```

5.4 Проверка и сохранение результатов

```

    final_population = last(results.numerical)
expected_population = last(results.analytical)
relative_error = abs(final_population - expected_population) / expected_population
time_to_double = doubling_time(alpha)

CSV.write(datadir(script_name, "trajectory.csv"), results)
jldsave(
    datadir(script_name, "all_results.jld2");
    parameters=(u0=u0, alpha=alpha, tspan=tspan, saveat=saveat),
    results=results,
    relative_error=relative_error,
)

trajectory_plot = plot(
    results.time,
    results.numerical;
    label="Численное решение",
    xlabel="Время, t",
    ylabel="Величина, u(t)",
    title="Экспоненциальный рост ( $\alpha = \alpha$ )",
    linewidth=3,
    legend=:topleft,
    size=(900, 540),
    dpi=150,
)
plot!(
    trajectory_plot,
    results.time,
    results.analytical;
    label="Аналитическое решение",
    linestyle=:dash,
    linewidth=2,
)
savefig(trajectory_plot, plotsdir(script_name, "exponential_growth.png"))

println("Экспоненциальный рост: одиночный эксперимент")
println(" u(0) = $(u0),  $\alpha = \alpha$ , t  $\in$  $(tspan)")
println(" u(10) численно    = $(round(final_population; digits=6))")

```

```
println(" u(10) аналитически = $(round(expected_population; digits=6))")
println(" относительная ошибка = $(round(relative_error; sigdigits=4))")
println(" время удвоения      = $(round(time_to_double; digits=4))")
println("Результаты: data/${script_name}, plots/${script_name}")
```

Полученная численная траектория совпадает с аналитической с малой относительной ошибкой, что подтверждает корректность реализации.

Листинг 2 — Полный сценарий базового эксперимента, интегрированный из `project/markdown/01_exponential_growth/01_exponential_growth.qmd`.

Получено 101 сохранённое значение. При $t = 10$ численное решение равно 20,0854485, аналитическое — 20,0855369. Относительная ошибка

$$\varepsilon_{rel} = \frac{|u_{num}(10) - u_{exact}(10)|}{|u_{exact}(10)|} \approx 4,40 \cdot 10^{-6}.$$

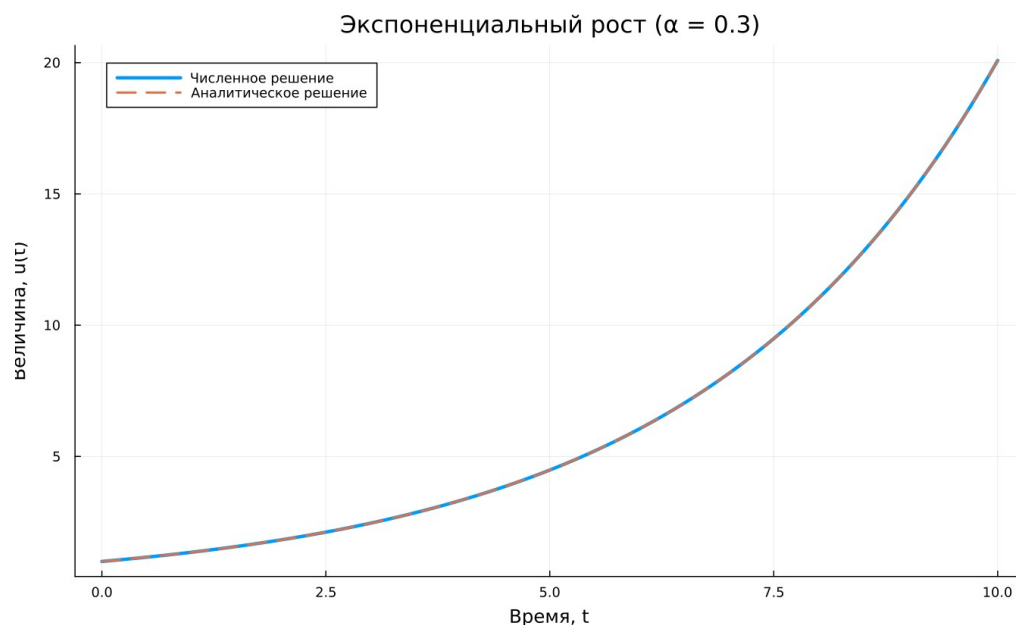


Рисунок 4: Численное и аналитическое решения при $\alpha = 0,3$

Кривые визуально совпадают, а малая относительная ошибка подтверждает корректность реализации.

5.5 Параметрическое исследование

Во втором сценарии коэффициент роста принимает значения 0,1, 0,3, 0,5, 0,8 и 1,0. Для каждого набора параметров `DrWatson.produce_or_load` сохраняет

отдельный результат. Такой способ позволяет расширять сетку параметров без дублирования вычислительного кода.

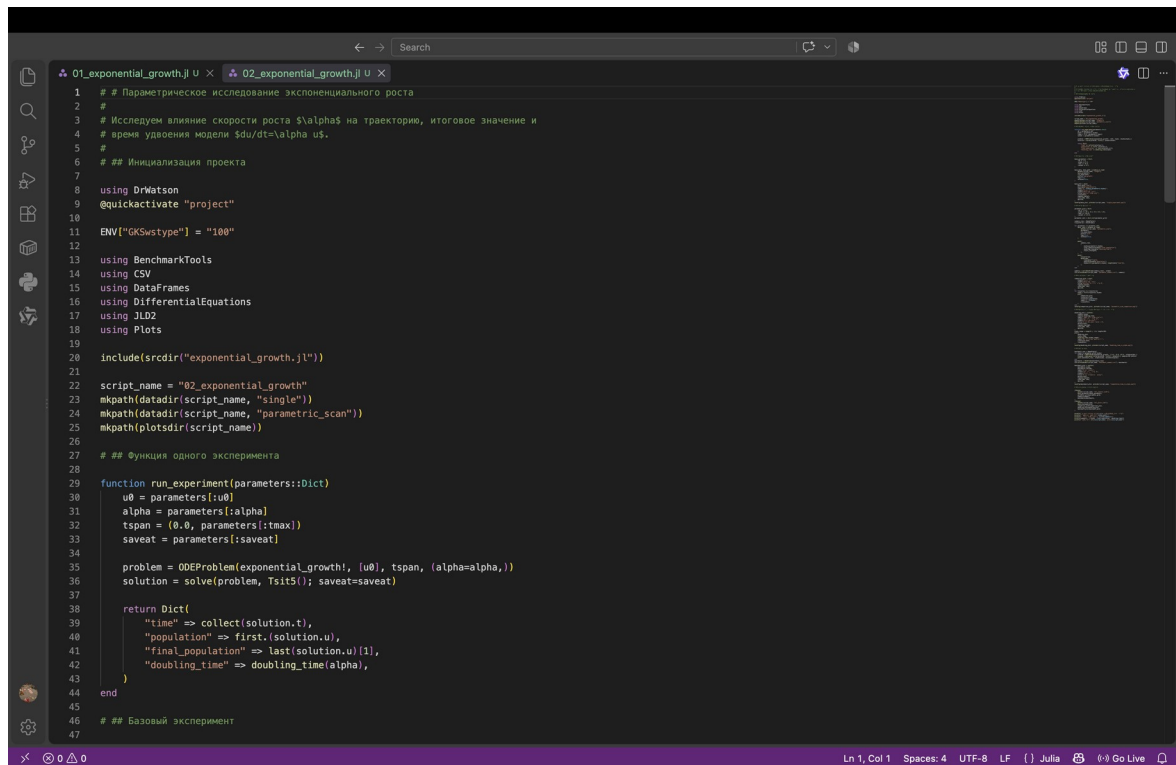


Рисунок 5: Фрагмент сценария серии экспериментов

Следующий фрагмент подключён из QMD, сформированного Literate.jl на основе scripts/02_exponential_growth.jl. Приведён весь сценарий: функция одного опыта, базовый запуск через produce_or_load, сетка параметров, обход пяти наборов, сохранение таблиц, четыре графика, бенчмарк и архивирование объектов.

6. Параметрическое исследование экспоненциального роста

Исследуем влияние скорости роста α на траекторию, итоговое значение и время удвоения модели $du/dt = \alpha u$.

6.1 Инициализация проекта

```
using DrWatson
@quickactivate "project"

ENV["GKSwstype"] = "100"

using BenchmarkTools
```

```

using CSV
using DataFrames
using DifferentialEquations
using JLD2
using Plots

include(scrdir("exponential_growth.jl"))

script_name = "02_exponential_growth"
mkpath(datadir(script_name, "single"))
mkpath(datadir(script_name, "parametric_scan"))
mkpath(plotsdir(script_name))

```

6.2 Функция одного эксперимента

```

function run_experiment(parameters::Dict)

    u0 = parameters[:u0]
    alpha = parameters[:alpha]
    tspan = (0.0, parameters[:tmax])
    saveat = parameters[:saveat]

    problem = ODEProblem(exponential_growth!, [u0], tspan, (alpha=alpha,))
    solution = solve(problem, Tsit5(); saveat=saveat)

    return Dict(
        "time" => collect(solution.t),
        "population" => first(solution.u),
        "final_population" => last(solution.u)[1],
        "doubling_time" => doubling_time(alpha),
    )
end

```

6.3 Базовый эксперимент

```

    base_parameters = Dict(
        :u0 => 1.0,
        :alpha => 0.3,
        :tmax => 10.0,
        :saveat => 0.1,
    )

    base_data, base_path = produce_or_load(
        datadir(script_name, "single"),
        base_parameters,
        run_experiment;

```

```

    prefix="exp_growth",
    tag=false,
    verbose=false,
)

base_plot = plot(
    base_data["time"],
    base_data["population"];
    label="α = $(base_parameters[:alpha])",
    xlabel="Время, t",
    ylabel="Величина, u(t)",
    title="Базовый эксперимент",
    linewidth=3,
    legend=:topleft,
    size=(900, 540),
    dpi=150,
)

savefig(base_plot, plotsdir(script_name, "single_experiment.png"))

```

6.4 Сетка параметров

```

    parameter_grid = Dict(
        :u0 => [1.0],
        :alpha => [0.1, 0.3, 0.5, 0.8, 1.0],
        :tmax => [10.0],
        :saveat => [0.1],
    )

parameter_sets = dict_list(parameter_grid)

summary_rows = NamedTuple[]
trajectories = DataFrame[]

for parameters in parameter_sets
    data, path = produce_or_load(
        datadir(script_name, "parametric_scan"),
        parameters,
        run_experiment;
        prefix="scan",
        tag=false,
        verbose=false,
    )

    push!(
        summary_rows,
        (

```

```

        alpha=parameters[:alpha],
        final_population=data["final_population"],
        doubling_time=data["doubling_time"],
        result_file=path,
    ),
)
push!(
    trajectories,
    DataFrame(
        time=data["time"],
        population=data["population"],
        alpha=fill(parameters[:alpha], length(data["time"])),
    ),
)
end

summary = sort(DataFrame(summary_rows), :alpha)
CSV.write(datadir(script_name, "parameter_summary.csv"), summary)

```

6.5 Сравнение траекторий

```

        comparison_plot = plot(
            xlabel="Время, t",
            ylabel="Величина, u(t)",
            title="Влияние скорости роста  $\alpha$ ",
            legend=:topleft,
            size=(900, 540),
            dpi=150,
        )
for trajectory in trajectories
    alpha = first(trajectory.alpha)
    plot!(
        comparison_plot,
        trajectory.time,
        trajectory.population;
        label=" $\alpha = \$(\alpha)$ ",
        linewidth=2,
    )
end
savefig(comparison_plot, plotsdir(script_name, "parametric_scan_comparison.png"))

```

6.6 Зависимость времени удвоения от скорости роста

```

doubling_plot = scatter(
    summary.alpha,
    summary.doubling_time;
    label="Результаты экспериментов",
    xlabel="Скорость роста,  $\alpha$ ",

```

```

ylabel="Время удвоения",
title="Время удвоения:  $\ln(2) / \alpha$ ",
markersize=7,
legend=:topright,
size=(900, 540),
dpi=150,
)
alpha_range = range(0.1, 1.0; length=100)
plot!(
    doubling_plot,
    alpha_range,
    doubling_time.(alpha_range);
    label="Теоретическая зависимость",
    linestyle=:dash,
    linewidth=2,
)
savefig(doubling_plot, plotsdir(script_name, "doubling_time_vs_alpha.png"))

```

6.7 Бенчмаркинг

```

benchmark_rows = NamedTuple[]
for alpha in parameter_grid[:alpha]
    problem = ODEProblem(exponential_growth!, [1.0], (0.0, 10.0), (alpha=alpha,))
    elapsed = @belapsed solve($problem, Tsit5(); saveat=0.1) samples=100 evals=1
    push!(benchmark_rows, (alpha=alpha, seconds=elapsed))
end
benchmarks = DataFrame(benchmark_rows)
CSV.write(datadir(script_name, "benchmark_summary.csv"), benchmarks)

benchmark_plot = scatter(
    benchmarks.alpha,
    benchmarks.seconds;
    label="Время решения",
    xlabel="Скорость роста,  $\alpha$ ",
    ylabel="Время, с",
    title="Время численного решения",
    markersize=7,
    legend=:topleft,
    size=(900, 540),
    dpi=150,
)
savefig(benchmark_plot, plotsdir(script_name, "computation_time_vs_alpha.png"))

```

6.8 Сохранение сводных данных

```

jldsave(
    datadir(script_name, "all_results.jld2");
    base_parameters=base_parameters,

```

```

parameter_grid=parameter_grid,
summary=summary,
benchmarks=benchmarks,
)
jldsave(
    datadir(script_name, "all_plots.jld2");
    base_plot=base_plot,
    comparison_plot=comparison_plot,
    doubling_plot=doubling_plot,
    benchmark_plot=benchmark_plot,
)

println("Параметрическое исследование экспоненциального роста")
println(" базовый результат: $(base_path)")
println(" число экспериментов: $(nrow(summary))")
println(summary[:, [:alpha, :final_population, :doubling_time]])
println("Результаты: data/${script_name}, plots/${script_name}")

```

Листинг 3 — Полный сценарий параметрического исследования, интегрированный из `project/markdown/02_exponential_growth/02_exponential_growth.qmd`.

Таблица 1: Результаты серии вычислительных экспериментов

α	$u(10)$	T_2
0,1	2,7183	6,9315
0,3	20,0854	2,3105
0,5	148,4086	1,3863
0,8	2 980,5736	0,8664
1,0	22 021,0156	0,6931

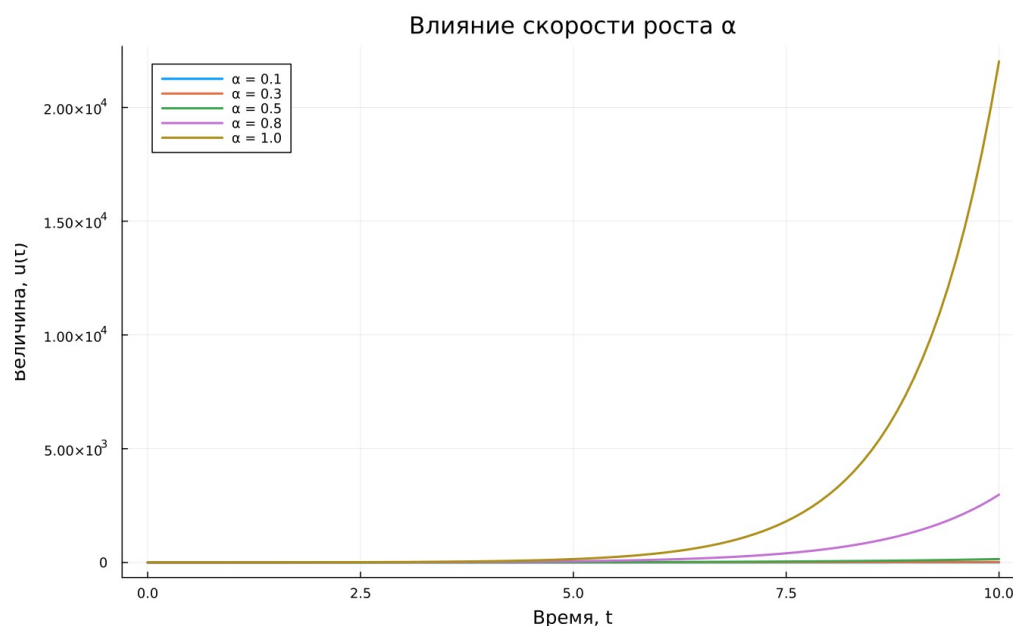


Рисунок 6: Сравнение траекторий для пяти коэффициентов роста

Итоговое значение экспоненциально зависит от α : увеличение коэффициента с 0,1 до 1,0 приводит к росту $u(10)$ примерно в 8102 раза. При этом время удвоения уменьшается обратно пропорционально α .

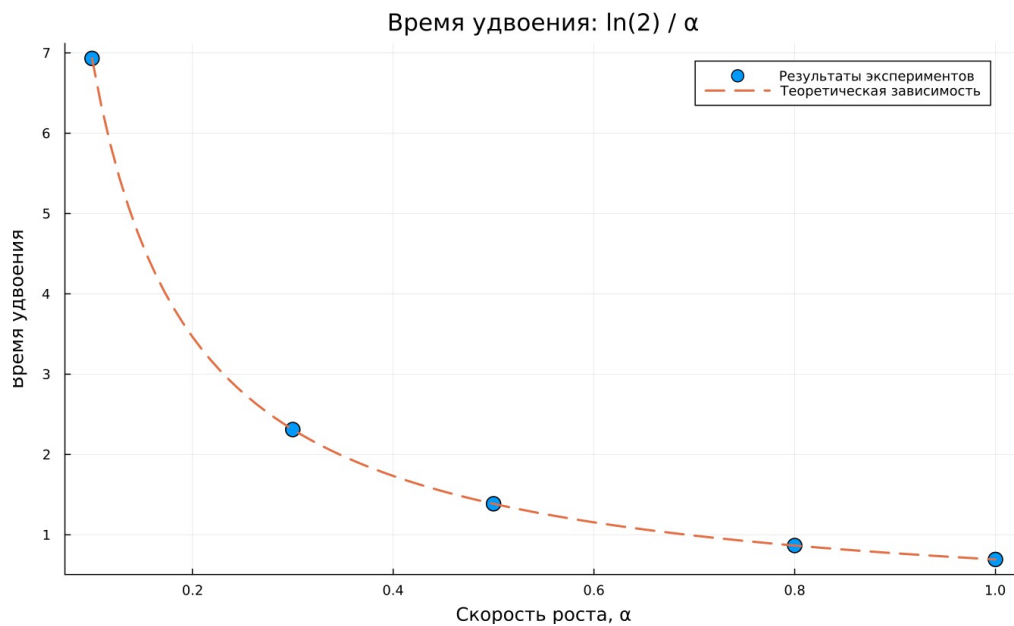


Рисунок 7: Расчётное время удвоения и теоретическая зависимость

Измерения `BenchmarkTools.jl` дают микросекундный порядок времени одного решения. Различия между точками невелики, поскольку интервал интегрирования, метод и сетка сохранения одинаковы.

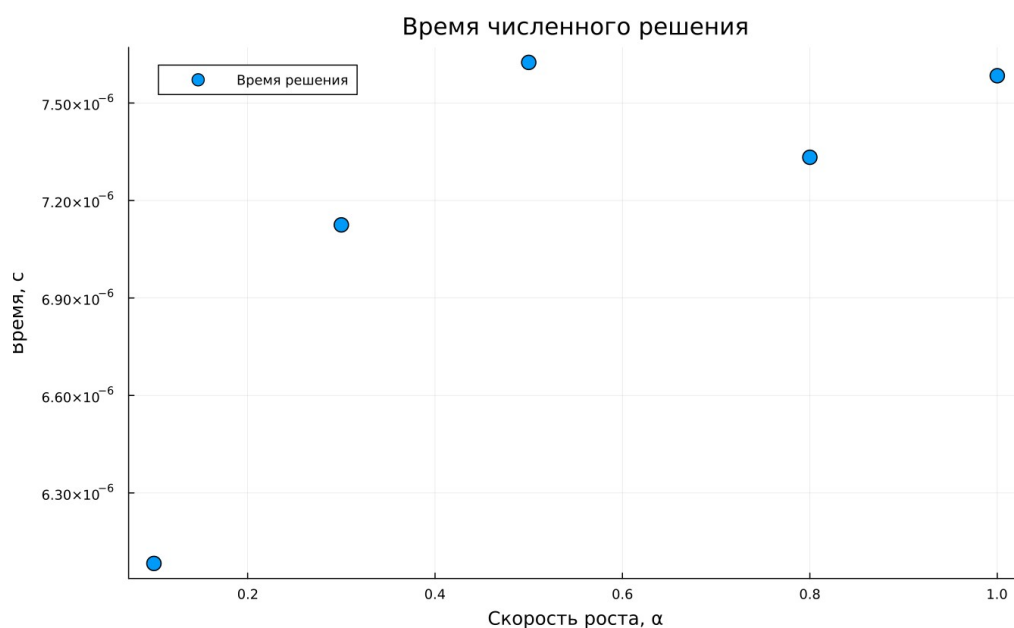


Рисунок 8: Измерение времени численного решения

7. Литературное программирование

Файл `scripts/tangle.jl` преобразует каждый исходный литературный сценарий в три согласованных представления:

- чистый .jl для пакетного запуска;
- .ipynb для интерактивной проверки;
- .qmd для публикации читаемой документации.

```
#!/usr/bin/env julia
```

```
using DrWatson
```

```
@quickactivate "project"
```

```
using Literate
```

```
function generate_formats(source::AbstractString)
```

```
    isfile(source) || error("Source file not found: $(source)")
```

```
    script_name = splitext(basename(source))[1]
```

```
    clean_dir = scriptsdir(script_name)
```

```
    quarto_dir = projectdir("markdown", script_name)
```

```
    notebook_dir = projectdir("notebooks", script_name)
```

```
    mkpath(clean_dir)
```

```
    mkpath(quarto_dir)
```

```
    mkpath(notebook_dir)
```

```
    println("Generating formats from $(source)")
```

```
    Literate.script(source, clean_dir; name=script_name, credit=false)
```

```
    println("  clean Julia: $(joinpath(clean_dir, script_name * ".jl"))")
```

```
    Literate.notebook(
```

```
        source,
```

```
        notebook_dir;
```

```
        name=script_name,
```

```
        execute=false,
```

```
        credit=false,
```

```
    )
```

```
    println("  notebook:  $(joinpath(notebook_dir, script_name * ".ipynb"))")
```

```
    Literate.markdown(
```

```
        source,
```

```
        quarto_dir;
```

```
        name=script_name,
```

```
        flavor=Literate.QuartoFlavor(),
```

```
        credit=false,
```

```
    )
```

```
    println("  Quarto:    $(joinpath(quarto_dir, script_name * ".qmd"))")
```

```
end
```

```
isempty(ARGS) && error(
    "Usage: julia --project=. scripts/tangle.jl <source.jl> [source.jl ...]",
)
foreach(generate_formats, ARGS)
```

Листинг 4 — Полный генератор производных форматов `scripts/tangle.jl`.

Оба ноутбука выполнены, поэтому выходные таблицы и графики сохранены внутри IPYNB и доступны после открытия без повторного расчёта.

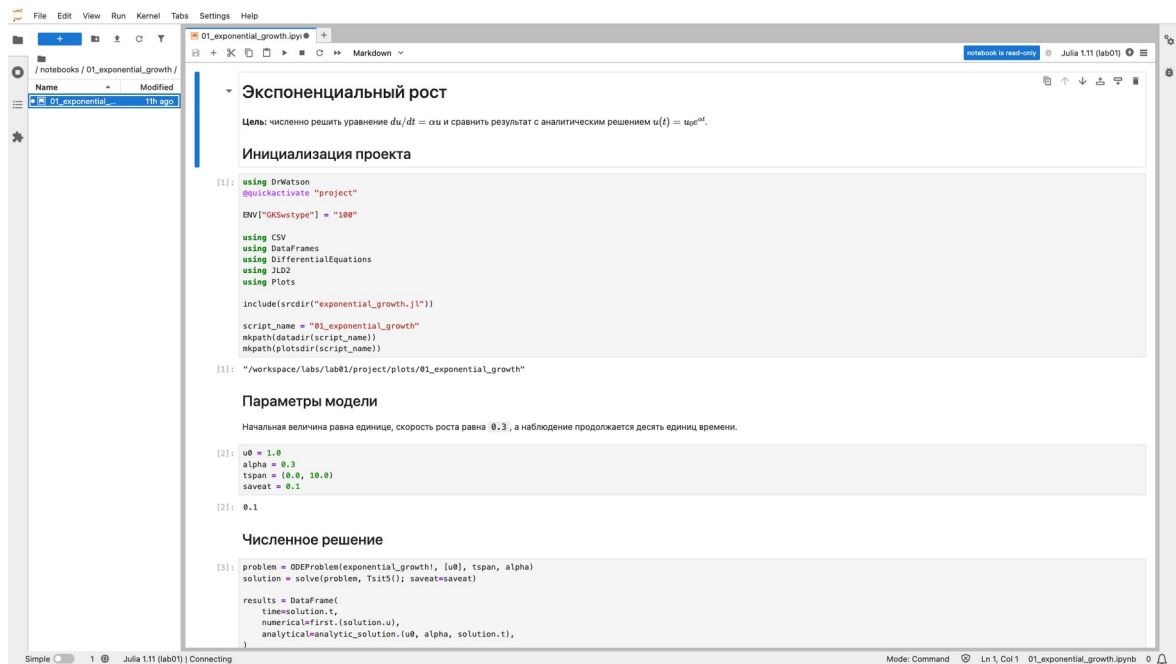


Рисунок 9: Выполненный ноутбук базового эксперимента

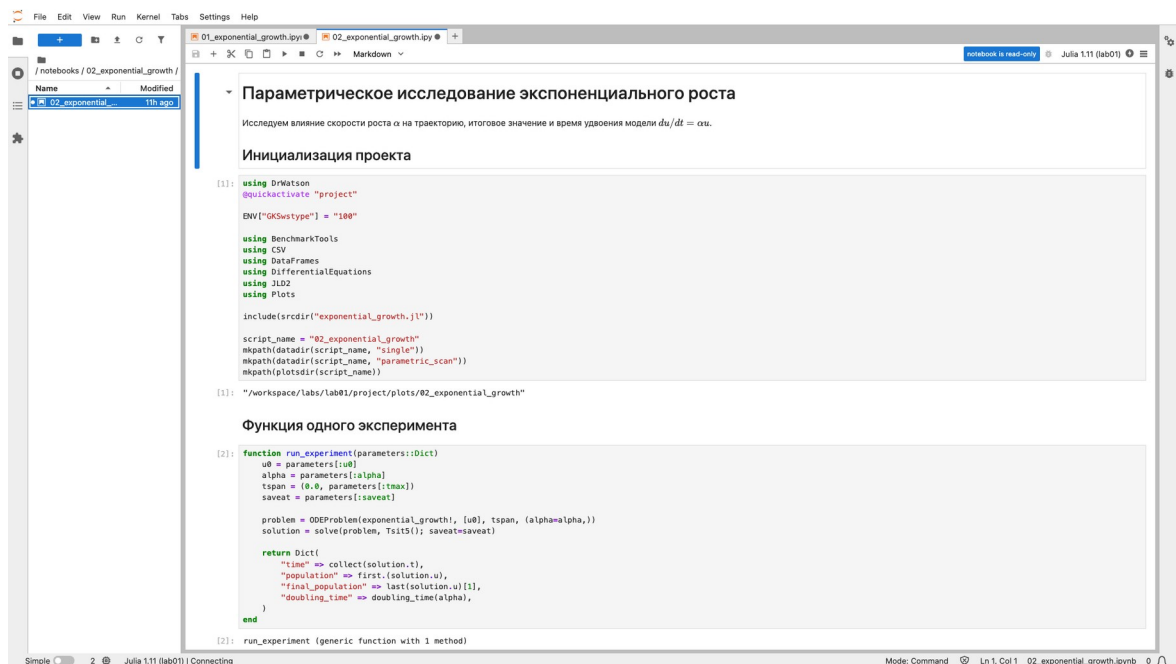


Рисунок 10: Выполненный ноутбук параметрического исследования

8. Проверка и воспроизводимость

Автоматические тесты проверяют четыре свойства: значение правой части ОДУ, аналитическую формулу, время удвоения и обработку недопустимого α .

```
using Test
using DifferentialEquations

include(joinpath(@__DIR__, "..", "src", "exponential_growth.jl"))

@testset "Exponential growth" begin
    u0 = 1.0
    alpha = 0.3
    tspan = (0.0, 10.0)
    problem = ODEProblem(exponential_growth!, [u0], tspan, alpha)
    solution = solve(problem, Tsit5(); saveat=0.1)

    @test isapprox(
        last(solution.u)[1],
        analytic_solution(u0, alpha, last(solution.t));
        rtol=1e-5,
    )

    @test isapprox(
        doubling_time(alpha),
```

```

2.3104906018664844;
rtol=1e-12,
)
@test_throws ArgumentError doubling_time(0.0)
end

println("All lab01 tests passed")

```

Листинг 5 — Полный набор автоматических проверок `test/runtests.jl`.

```

rhktrlwmd@Mac 2026-1--study--simulation-modeling % docker exec lab01-julia make test
julia --project=. test/runtests.jl
Test Summary: | Pass Total Time
Exponential growth | 3 3 1.1s
All lab01 tests passed
rhktrlwmd@Mac 2026-1--study--simulation-modeling % 

```

Рисунок 11: Успешное выполнение автоматических тестов

Сборка управляется целями Makefile. Последовательность `tangle`, `run`, `notebooks`, `docs`, `test` восстанавливает производные форматы, результаты и документацию. Отдельные Makefile формируют отчёт и презентацию.

```

rhktrlwmd@Mac 2026-1--study--simulation-modeling % docker exec lab01-julia find data plots notebooks markdown -type f
data/02_exponential_growth/single/exp_growth_alpha=0.3_saveat=0.1_tmax=10.0_u0=1.0.jld2
data/02_exponential_growth/parameter_summary.csv
data/02_exponential_growth/parametric_scan/scan_alpha=0.1_saveat=0.1_tmax=10.0_u0=1.0.jld2
data/02_exponential_growth/parametric_scan/scan_alpha=0.5_saveat=0.1_tmax=10.0_u0=1.0.jld2
data/02_exponential_growth/parametric_scan/scan_alpha=0.3_saveat=0.1_tmax=10.0_u0=1.0.jld2
data/02_exponential_growth/parametric_scan/scan_alpha=1.0_saveat=0.1_tmax=10.0_u0=1.0.jld2
data/02_exponential_growth/parametric_scan/scan_alpha=0.8_saveat=0.1_tmax=10.0_u0=1.0.jld2
data/02_exponential_growth/benchmark_summary.csv
data/02_exponential_growth/all_plots.jld2
data/02_exponential_growth/all_results.jld2
data/01_exponential_growth/trajectory.csv
data/01_exponential_growth/all_results.jld2
plots/02_exponential_growth/doubling_time_vs_alpha.png
plots/02_exponential_growth/computation_time_vs_alpha.png
plots/02_exponential_growth/single_experiment.png
plots/02_exponential_growth/parametric_scan_comparison.png
plots/01_exponential_growth/exponential_growth.png
notebooks/02_exponential_growth/02_exponential_growth.ipynb
notebooks/01_exponential_growth/01_exponential_growth.ipynb
markdown/02_exponential_growth/02_exponential_growth.qmd
markdown/02_exponential_growth/02_exponential_growth.html
markdown/01_exponential_growth/01_exponential_growth.html
markdown/01_exponential_growth/01_exponential_growth.qmd
rhktrlwmd@Mac 2026-1--study--simulation-modeling % []

```

Рисунок 12: Проверка созданных данных, графиков и документов

Для повторения работы на другом компьютере достаточно установить Julia и Quarto, клонировать репозиторий, восстановить зависимости и **ВЫПОЛНИТЬ:**

```

cd labs/lab01/project
julia --project=. -e 'using Pkg; Pkg.instantiate()'
make all

```

Полная последовательность целей проекта задана следующим Makefile.

```

.PHONY: setup tangle run notebooks docs test all clean-results

setup:
    julia --project=. scripts/setup_environment.jl

tangle:
    julia --project=. scripts/tangle.jl scripts/01_exponential_growth.jl
    julia --project=. scripts/tangle.jl scripts/02_exponential_growth.jl

run:
    julia --project=. scripts/01_exponential_growth.jl
    julia --project=. scripts/02_exponential_growth.jl

notebooks:

```

```
jupyter nbconvert --to notebook --execute --inplace \
  --ExecutePreprocessor.kernel_name=julia-lab01-1.11 \
  --ExecutePreprocessor.timeout=600 \
  notebooks/01_exponential_growth/01_exponential_growth.ipynb
jupyter nbconvert --to notebook --execute --inplace \
  --ExecutePreprocessor.kernel_name=julia-lab01-1.11 \
  --ExecutePreprocessor.timeout=600 \
  notebooks/02_exponential_growth/02_exponential_growth.ipynb
```

docs:

```
quarto render markdown/01_exponential_growth/01_exponential_growth.qmd \
  --to html --embed-resources
quarto render markdown/02_exponential_growth/02_exponential_growth.qmd \
  --to html --embed-resources
```

test:

```
julia --project=. test/runtests.jl
```

all: tangle run notebooks docs test

clean-results:

```
rm -rf data/01_exponential_growth data/02_exponential_growth
rm -rf plots/01_exponential_growth plots/02_exponential_growth
rm -rf notebooks/01_exponential_growth notebooks/02_exponential_growth
rm -rf markdown/01_exponential_growth markdown/02_exponential_growth
```

Листинг 6 — Полный project/Makefile.

Начальная настройка среды выполняется отдельным скриптом.

```
using Pkg

Pkg.activate(joinpath(@__DIR__, ".."))
Pkg.instantiate()
Pkg.precompile()

using IJulia

project_path = dirname(Base.active_project())
IJulia.installkernel("Julia lab01", "--project=$(project_path)")

println("Environment is ready")
```

```
println("Julia: ", VERSION)
println("Project: ", project_path)
```

Листинг 7 — Полный скрипт `scripts/setup_environment.jl`.

Файл `Project.toml` фиксирует имя проекта, версию Julia и прямые зависимости. Точные транзитивные версии записаны в `Manifest.toml`.

```
name = "project"
uuid = "5e6fc9a5-189d-46e2-b824-9dd395edfa0f"
authors = ["rhktrlwmd"]
version = "1.0.0"

[deps]
BenchmarkTools = "6e4b80f9-dd63-53aa-95a3-0cdb28fa8baf"
CSV = "336ed68f-0bac-5ca0-87d4-7b16caf5d00b"
DataFrames = "a93c6f00-e57d-5684-b7b6-d8193f3e46c0"
DifferentialEquations = "0c46a032-eb83-5123-abaf-570d42b7fbba"
DrWatson = "634d3b9d-ee7a-5ddf-bec9-22491ea816e1"
IJulia = "7073ff75-c697-5162-941a-fcdaad2a7d2a"
JLD2 = "033835bb-8acc-5ee8-8aae-3f567f8a3819"
Literate = "98b081ad-f1c9-55d3-8b20-4c87d4299306"
Plots = "91a5bcdd-55d7-5caf-9e0b-520d859cae80"
Quarto = "d7167be5-f61b-4dc9-b75c-ab62374668c5"

[compat]
julia = "1.11"
```

Листинг 8 — Полный `project/Project.toml`.

9. Учёт изменений

История изменений описана в двух уровнях: общий `CHANGELOG.md` фиксирует релизы курса, а `labs/lab01/CHANGELOG.md` содержит проверяемые результаты первой лабораторной. Публикация помечается тегом `lab01`.

```
rhktrlwmd@Mac 2026-1--study--simulation-modeling % git log --oneline --decorate --graph --all
* d9f59b4 (HEAD -> main) feat(lab01): complete exponential growth study
* 098f9e1 chore: initialize simulation modeling course structure
rhktrlwmd@Mac 2026-1--study--simulation-modeling % █
```

Рисунок 13: История версии лабораторной работы

```
rhktrlwmd@Mac 2026-1--study--simulation-modeling % find labs/lab01/{report,presentation} -maxdepth 1 -type f | sort
labs/lab01/presentation/.gitignore
labs/lab01/presentation/Makefile
labs/lab01/presentation/_quarto.yml
labs/lab01/presentation/simulation-modeling-lab01-presentation.html
labs/lab01/presentation/simulation-modeling-lab01-presentation.pdf
labs/lab01/presentation/simulation-modeling-lab01-presentation.qmd
labs/lab01/presentation/styles.css
labs/lab01/report/.gitignore
labs/lab01/report/Makefile
labs/lab01/report/_quarto.yml
labs/lab01/report/simulation-modeling-lab01-report.docx
labs/lab01/report/simulation-modeling-lab01-report.pdf
labs/lab01/report/simulation-modeling-lab01-report.qmd
rhktrlwmd@Mac 2026-1--study--simulation-modeling % █
```

Рисунок 14: Состав материалов релиза lab01

10. Выводы

В работе подготовлен воспроизводимый Julia-проект и реализована модель экспоненциального роста. Для базового набора параметров численное решение согласуется с аналитическим с относительной ошибкой около $4,40 \cdot 10^{-6}$. Серия из пяти экспериментов показала резкую зависимость итогового значения от α и подтвердила формулу времени удвоения.

Литературный исходный код стал единым источником для чистых программ, ноутбуков и Quarto-документации. Данные, графики, выполненные IPYNB, тесты, отчёт и презентация организованы в одной структуре и могут быть повторно получены командами проекта.

Сопоставление численного и аналитического решений на всей исследованной сетке времени дополнительно подтверждает корректность программной реализации.

Источники

1. *Имитационное моделирование: методические указания к лабораторным работам.* Раздел 1.10 «Модель экспоненциального роста». С. 70–86.
2. Rackauckas C., Nie Q. DifferentialEquations.jl — A Performant and Feature-Rich Ecosystem for Solving Differential Equations in Julia. Journal of Open Research Software. 2017. Vol. 5. No. 1.
3. DrWatson.jl Documentation. <https://juliadynamics.github.io/DrWatson.jl/>.
4. Literate.jl Documentation. <https://fredrikekre.github.io/Literate.jl/>.
5. Quarto Documentation. <https://quarto.org/docs/>.